月墨（MoonInk）项目申报书

一、项目基本信息

1. 项目名称

中文名称：月墨
英文名称：MoonInk

2. 项目一句话简介

月墨（MoonInk）是一个使用 MoonBit 构建的静态站点生成器，面向文档、博客与知识站点，首版聚焦于个人知识发布与结构化内容组织场景。

3. 项目定位

月墨定位为一个内容优先、结构清晰、行为可预测的静态站点生成工具。它强调以较低的配置复杂度，将 Markdown 内容组织为可导航、可搜索、可长期维护的知识型站点，同时作为 MoonBit 在真实工程级开发工具场景中的示范项目。

二、项目目标与应用场景

1. 项目目标

本项目拟实现一个基于 MoonBit 的静态站点生成器，完成从内容输入、结构组织、链接解析、主题渲染到站点输出的完整工具链闭环。项目目标主要包括以下几个方面：

- 构建一个面向知识发布场景的静态站点生成器，使用户能够以 Markdown 为主要内容载体进行站点构建。
- 验证 MoonBit 在配置解析、内容建模、静态渲染、命令行工具开发等工程型任务中的适用性。
- 形成一套结构清晰、边界明确、可测试、可演进的工程架构，为后续扩展主题系统、插件机制、多语言能力和部署能力奠定基础。
- 输出一套完整的设计文档、路线图和工程技术文档，支撑项目申报、后续实现与长期维护。

2. 拟解决的实际问题

当前文档站点、知识站点和个人知识发布通常依赖已有生态成熟的静态站点生成器，例如 MkDocs、Hugo、Docusaurus 等。这些工具已经证明了“Markdown + 配置 + 模板渲染 + 静态发布”这一模式的有效性，但在 MoonBit 生态中，尚缺少一个能够完整展示语言工程能力、同时具备实际可用价值的同类项目。

本项目拟解决的问题包括：

- MoonBit 缺乏一个具有代表性的真实开发工具项目，难以充分展示其在复杂工程系统中的能力。

- 个人知识发布场景常同时存在“层级文档组织”和“文章型内容流”两种需求，现有很多方案要么偏文档站，要么偏博客，知识库式双轨结构支持不够自然。
- 知识型内容中常出现大量内部链接、引用关系和跨页面跳转，普通 Markdown 链接管理成本较高，且断链、歧义链接等问题不易及时发现。
- 从内容写作到站点发布的过程需要一个可预测、低心智负担的构建工具，以减少维护成本并提升长期可持续性。

3. 应用场景

本项目首版主要面向以下应用场景：

- 个人知识库站点：用于发布结构化笔记、学习记录、专题整理与长期知识沉淀。
- 项目文档站点：用于生成层次清晰的说明文档、教程、指南和参考资料。
- 轻量博客与知识混合站点：一部分内容按章节组织，一部分内容按时间与标签组织。
- MoonBit 生态示范项目：作为 MoonBit 开发工具链能力与工程实践能力的展示样例。

三、交付物说明

1. 核心交付物

项目的主要交付物包括：

- 一个基于 MoonBit 实现的静态站点生成器原型；
- 命令行工具 `moonink`；
- 一套可运行的默认主题；
- 配套的 RFC、Roadmap 和技术文档；
- 面向实现和使用的中文文档说明；
- 用于演示与测试的样例站点结构。

2. 拟支持的核心功能

首版计划支持以下核心功能：

- 基于 TOML 的项目配置；
- 基于 Markdown 的内容输入；
- 支持带 frontmatter 的页面元数据；
- 支持双轨内容结构，即文档树与文章流并存；
- 自动生成默认导航，并支持手工调整；
- 支持内部链接校验；
- 支持 WikiLink 风格链接语法；
- 生成本地静态搜索索引；
- 提供默认官方主题；
- 提供项目初始化、构建与本地预览命令。

3. 功能边界（Scope）

为保证首版实现可控，项目将明确限制以下内容不进入第一阶段交付：

- 不追求与 MkDocs 配置或插件体系完全兼容；
- 不开放第三方插件机制；

- 不实现多主题分发与主题市场；
- 不实现完整多语言站点支持；
- 不实现增量构建缓存；
- 不实现可视化知识图谱；
- 不实现复杂的内容组件语法或类似 MDX 的嵌入式交互内容。

4. 预期使用方式与交互流程

预期的基本使用流程如下：

1. 用户通过 `moonink new` 初始化一个新项目；
2. 在项目目录中配置站点信息与内容目录；
3. 使用 Markdown 编写文档页与文章页，并通过 frontmatter 描述页面元数据；
4. 使用 `moonink build` 执行内容扫描、解析、链接校验、导航构建、页面渲染与静态输出；
5. 使用 `moonink serve` 在本地预览生成结果；
6. 将输出目录部署到任意静态托管平台。

5. 初步测试规划

测试将以单元测试为基础，并逐步引入基于样例站点的回归校验。首版测试重点包括：

- 配置解析与配置校验；
- frontmatter 解析；
- Markdown 内容解析边界；
- 页面路由生成；
- 导航生成；
- 内部链接校验；
- WikiLink 解析；
- 搜索索引生成。

后续阶段可逐步增加：

- 基于 fixture 的构建回归测试；
- 输出结果快照比对；
- 本地预览与构建流程集成测试。

6. 文档与使用说明覆盖范围

项目文档将覆盖以下内容：

- 项目背景、目标与范围说明；
- 架构设计与模块划分；
- 配置文件说明；
- 内容模型与页面元数据说明；
- 路由、导航、链接与 WikiLink 规则说明；
- 主题系统说明；
- CLI 使用方式；
- 测试策略与质量保障说明；
- 风险分析与后续演进方向。

四、技术路线说明

1. 整体技术路线

本项目采用“静态内容处理流水线”作为核心技术路线，从内容到站点输出的过程分为若干清晰阶段：

1. 加载并校验项目配置；
2. 发现内容文件与静态资源；
3. 解析 frontmatter 与 Markdown；
4. 规范化页面标识与输出路由；
5. 解析内部链接与 WikiLink；
6. 构造站点模型与导航结构；
7. 通过默认主题渲染 HTML；
8. 输出静态页面、资源文件与搜索索引。

该技术路线的特点是模块边界清晰、阶段职责明确、可测试性较强，并适合后续逐步引入扩展点。

2. 整体系统架构

系统总体上划分为以下核心层次：

- CLI 层：负责命令解析、执行入口与用户交互反馈；
- 配置层：负责 TOML 配置解析、类型校验与默认值处理；
- 内容层：负责扫描内容目录、识别文件类型、加载文本内容；
- 解析层：负责 frontmatter 解析、Markdown 解析、标题提取与内容规范化；
- 链接层：负责内部链接解析、WikiLink 解析、目标校验与冲突诊断；
- 路由与导航层：负责生成页面 URL、构建文档树与文章流导航；
- 站点模型层：负责统一组织页面、导航、标签、搜索数据等全站信息；
- 渲染层：负责主题模板渲染、页面 HTML 生成与静态资源注入；
- 输出层：负责写入构建结果、复制资源、生成搜索索引。

3. 核心模块划分

计划中的核心模块包括：

- `cli`
- `config`
- `content`
- `frontmatter`
- `markdown`
- `links`
- `routing`
- `nav`
- `model`
- `theme`
- `render`
- `search`

- `serve`

这些模块对应不同处理阶段，尽量保持单一职责，避免后续演进时出现结构性耦合。

4. 核心设计要点

本项目在技术设计上有几个关键选择：

- 配置格式采用 TOML，而非 YAML。这样有利于保持结构清晰和解析稳定。
- 内容格式第一版仅支持 Markdown，以控制范围并保证实现质量。
- 内容组织采用双轨结构，同时支持文档树和文章流，贴合知识发布场景。
- 页面元数据采用较小的核心集合，包括 `title`、`id`、`author`、`tags`、`date`。
- 链接模型同时支持普通内部链接与 `[[WikiLink]]`。
- 主题系统第一版仅提供单一官方主题，保证样式与功能完整性。
- 插件体系在架构中预留位置，但不在首版开放。
- 搜索采用构建时生成静态索引、前端本地搜索的方案。

5. 关键技术选型说明

本项目关键技术选型如下：

- 编程语言：MoonBit
- 配置格式：TOML
- 内容格式：Markdown
- 链接增强：WikiLink
- 搜索方案：静态搜索索引 + 前端本地检索
- 命令行工具形态：`new` / `build` / `serve`
- 主题策略：单官方主题
- 测试策略：单元测试优先，逐步扩展到 fixture 回归测试

6. 参考基础与实现支撑

在 Markdown 能力方面，MoonBit 已存在可参考或可复用的 Markdown parser 与 linter 实现，因此本项目的重点不在于从零构建 Markdown 解析器，而在于将 Markdown 解析、配置处理、链接校验、站点模型和页面渲染整合为一个完整的静态站点生成系统。

7. 大模型与智能体工具在开发过程中的作用

在项目设计与实现阶段，大模型与智能体工具可承担以下辅助作用：

- 参与需求收敛与功能边界讨论；
- 协助整理 RFC、Roadmap 与技术设计文档；
- 帮助梳理模块边界、风险点与实现路线；
- 辅助生成测试样例、配置样例与文档草稿；
- 协助对比参考项目的架构与功能设计；
- 为代码实现提供原型建议和重构思路。

在项目中，大模型与智能体工具的角色应定位为“设计与研发辅助工具”，而不是替代工程实现本身。其价值主要在于提升方案分析效率、文档组织效率和局部编码效率。

五、风险分析与应对方案

1. 技术风险

（1）MoonBit 生态成熟度风险

尽管 MoonBit 已具备一定基础能力，但在配置处理、模板渲染、文件监听、CLI 工具开发等工程辅助库方面，生态成熟度仍可能不如 Python、JavaScript 等传统工具语言。

应对方案：

- 保持首版范围收敛；
- 优先构建清晰的内部抽象，减少对外部不稳定依赖的强耦合；
- 在架构设计中保留后续替换实现的可能性。

（2）渲染与主题复杂度风险

主题系统和页面渲染机制容易从“能用”快速膨胀为“高度可配置”，从而显著增加工程复杂度。

应对方案：

- 第一版只实现一个官方主题；
- 将主题系统设计为内部可替换，但暂不开放稳定公共接口；
- 优先保证默认主题可用与一致，而不是追求主题生态。

（3）知识库场景范围膨胀风险

一旦支持 WikiLink 和知识组织，用户很容易继续期待反向链接、图谱、引用网络等功能，这会推动需求快速膨胀。

应对方案：

- 第一版只实现双轨内容结构、WikiLink 与搜索；
- 将图谱、反向链接、复杂引用能力放入后续路线图；
- 在申报阶段明确功能边界，避免过度承诺。

（4）本地预览与文件监听风险

`serve` 命令背后的文件监听、自动重建和刷新机制，往往比静态构建本身更复杂。

应对方案：

- 第一版先实现实用型本地预览；
- 从简单的“重建 + 刷新”模式起步；
- 将更高级热重载能力延后到后续阶段。

2. 工程风险

（1）架构过早抽象风险

如果在项目早期就为插件、主题生态或多语言支持设计过多抽象，容易导致实现困难、文档复杂、测试成本升高。

应对方案：

- 先完成核心生成器闭环；
- 将扩展能力设计为后续演进方向；
- 在不开放生态的前提下，先验证内部架构边界。

（2）测试覆盖不足风险

静态站点生成器涉及配置、内容、路由、渲染、搜索等多个阶段，如果测试不足，容易在后续迭代中引入回归问题。

应对方案：

- 以单元测试为首要质量基线；
- 使用样例站点与 fixture 做回归检查；
- 逐步扩展到更强的输出验证机制。

3. 替代方案与退路

若某些模块实现难度高于预期，可采用以下替代方案：

- 若主题层复杂度过高，可先将主题能力缩减为单模板体系；
- 若本地预览复杂度过高，可先交付仅构建不监听的基础版本，再补 `serve`；
- 若搜索索引设计过于复杂，可先交付基础关键字检索，再逐步优化排序和索引粒度；
- 若某些高级链接校验难以稳定实现，可先保证路径级链接与 WikiLink 主流程可用。

六、相关研究与实践基础

1. 相关开源项目与工程实践

本项目的设计参考了现有静态站点生成器的成熟经验，尤其是文档站点生成工具的以下共性：

- 使用 Markdown 作为内容源；
- 使用配置文件描述站点信息与构建策略；
- 使用主题模板将结构化内容渲染为 HTML；
- 支持导航、搜索、静态资源组织与部署输出；
- 通过 CLI 暴露项目初始化、构建与预览等基本工作流。

其中，以文档站为主要方向的开源项目提供了清晰的参考框架：如何处理导航、如何组织内容、如何定义构建输出，以及如何平衡“可用性”和“可扩展性”。

2. 对相关技术现状的理解

当前静态站点生成器技术已经较成熟，核心差异往往不在“能否生成 HTML”，而在以下方面：

- 内容模型是否清晰；
- 配置系统是否稳定；

- 路由和导航是否可预测；
- 搜索与链接能力是否完善；
- 主题系统是否合理；
- CLI 工作流是否顺畅；
- 架构是否便于长期维护和演进。

MoonInk 的目标不是简单重复已有项目，而是在 MoonBit 语言环境下，将这些成熟经验重新组织为一个适合知识型发布场景、边界清晰且工程上可解释的系统。

3. 本项目的实践基础

本项目已经完成了前期需求收敛、RFC 起草、路线图制定以及技术文档设计，明确了以下关键内容：

- 项目定位；
- 功能边界；
- 模块架构；
- 内容模型；
- 路由与导航策略；
- WikiLink 支持方向；
- 搜索、主题、CLI 和测试策略；
- 后续演进路线。

这意味着项目已经具备较扎实的方案基础，可在此基础上进入实现阶段。

七、个人背景与项目匹配度

1. 项目匹配度说明

本项目适合作为一个兼具工程实现价值与技术研究价值的开发工具项目。一方面，静态站点生成器本身具备明确的问题边界和成熟的参考范式；另一方面，其内部又包含配置解析、内容建模、链接处理、渲染输出、CLI 设计等多个可拆解的工程问题，非常适合系统性设计与逐步实现。

申报人（Kozmosa）

- 使用过 Markdown、文档工具链或静态站点生成器；
- 具有命令行工具、编译流程或构建系统相关经验；
- 熟悉内容发布、知识组织或文档写作场景；
- 能够持续进行模块化实现与文档维护。 与该项目的应用场景以及开发所需的技能基础较为契合

2. 与个人技术成长的关系

该项目不仅能够产出一个实际可用的工具原型，也有助于本人在以下方面形成系统性能力：

- 使用 MoonBit 开展工程级开发实践；
- 将产品需求转化为架构设计与模块划分；
- 通过测试与文档保障长期可维护性；
- 形成从需求分析到技术交付的完整项目经验；

- 使用 Agentic Engineering (instead of simply vibe coding) 完整构建项目；
- 有助于深入理解静态文档生成器的内部机制，助力对现有项目（survive-in-scut / MySCUT）的维护、重构与功能增强。

八、其他补充材料

1. 原型与文档材料

本项目已形成较完整的前期设计材料，可作为后续理解和实现的基础，这些材料涵盖了：

- RFC
  - 项目方向与核心决策；
  - 功能边界与非目标范围；
  - 架构与技术模块说明；
  - 风险分析与演进方向。
- Roadmap
  - 路线图与阶段划分；

2. 品牌与产品表达

项目品牌名称为“月墨（MoonInk）”，整体表达强调克制、清晰、精确、安静与可持续。这一品牌方向与项目的产品设计和工程风格保持一致：内容优先、结构明确、不过度追求炫技式复杂性。

3. 后续阶段预期

在首版完成后，项目可沿以下方向继续演进：

- 增量构建：热重载开发服务器，增量构建
- 多语言支持：增加对网站ui以及内容本身的i18n支持
- 部署集成：编写开箱即用的集成CI
- 更完善的检查工具：编写开箱即用的中英文Linter
- 主题扩展机制：增加主题扩展的自由度，增加更多主题预设
- 插件机制：增加更多插件API，让插件能够以低耦合、高鲁棒的方式接入生成器
- 反向链接与知识图谱增强能力：构建类似obsidian自动生成的知识图谱
- LLM-Friendly Site：为项目集成开箱即用的LLM索引、LLMs.txt/LLMs-full.txt生成功能

九、结语

月墨（MoonInk）项目的核心价值，在于使用 MoonBit 实现一个结构清晰、目标明确、具有真实应用场景的静态站点生成器。它既面向知识内容发布的实际需求，也承担了验证 MoonBit 工程能力的重要任务。

从项目本身看，月墨将通过 Markdown 内容组织、双轨信息结构、WikiLink、静态搜索、默认主题和 CLI 工作流，形成一个完整的最小可用闭环；从工程层面看，它将通过清晰的模块划分、分阶段构建流水线、测试规划与文档体系，形成一个具备持续实现与扩展基础的开发工具项目。

因此，本项目具备明确的问题指向、可执行的技术路线、合理的交付边界以及较好的后续演进空间，适合作为一个兼具实践价值与示范意义的申报项目。